# CS683 Programming Assignment II

Unassigned team 1

24b0957, 24b1041, 23b1012, 210050107

**Setup:** Single-core ChampSim, GCC 7.3, three supplied traces, 25M warmup and 25M measured instructions. Baseline means no prefetching and L2 LRU. Speedup is IPC divided by Baseline IPC; MPKI is load misses per 1,000 instructions. Accuracy is useful prefetches divided by prefetches sent to the next level; coverage is the percentage reduction in L1D load misses.

## Task 1: Stride Pattern Prefetcher

We extended the IP-stride idea by keeping 48 full-IP trackers, each with 32 pattern entries. The supplied code remembers one stride; our version also maps the last four strides to the next stride. We prefer a uniform stride at confidence 3 and otherwise use a pattern at confidence 2 or higher. Every predicted address stays in the current 4 KB page. We chose accuracy thresholds of 40% and 75% with degrees 1, 2, and 4: degree 1 limits wasted requests, degree 2 gives moderate lookahead, and degree 4 looks farther ahead when predictions are useful. The conservative and aggressive threshold choices were nearly tied; the larger tracker table and dispatched-prefetch feedback performed better. Each tracker's metadata uses IP 64, last line address 58, signature 28, last stride 7, confidence 2, history length 3, LRU rank 6, pattern pointer 5, and valid 1 bits: 174 bits. Each pattern uses signature 28, stride 7, confidence 2, and valid 1 bits: 38 bits. Total prediction/control storage is $48(174 + 32 \times 38) + 128 + 3 = 66{,}851$ bits. The actual C++ storage is 13,868 bytes per CPU, including padding and three diagnostic counters.

**Task 1 results**

| Policy | Trace | IPC | Speedup | L1D load MPKI | Accuracy (%) | Coverage (%) |
|---|---|---|---|---|---|---|
| Baseline | 1 | 0.232 | 1.000 | 10.68 | N/A | 0.00 |
| Baseline | 2 | 0.250 | 1.000 | 13.54 | N/A | 0.00 |
| Baseline | 3 | 0.382 | 1.000 | 3.40 | N/A | 0.00 |
| SPP | 1 | 0.233 | 1.005 | 10.23 | 65.14 | 4.18 |
| SPP | 2 | 0.251 | 1.002 | 13.42 | 65.56 | 0.89 |
| SPP | 3 | 0.383 | 1.002 | 3.34 | 30.66 | 1.78 |

## Task 2: I-RRIP and Dynamic I-RRIP

We use the instruction flag to distinguish instruction lines from data. We insert instructions at RRPV 1 and data at 2; instruction hits promote to 0 and data hits to at most 1. On eviction, we choose data first only among lines already at RRPV 3; this protects code without pinning it in the cache. These rules gave higher aggregate IPC than inserting instructions at 0 or promoting data hits to 0. For Dynamic I-RRIP, 16 of the 1,024 sets always use SRRIP and sample demand accesses and misses. Every 4,096 sampled accesses, we compare $(I_A+I_M)/(I_A+D_A+I_M+D_M)$ with threshold 0.25 and hysteresis 0.02; the middle range keeps the current mode (A counts accesses and M counts misses). Sixteen leaders gave the best tested aggregate result while leaving most sets free to follow the decision. The shorter and longer windows were worse, so we kept 4,096. Instruction misses fall, but data misses can rise; Dynamic I-RRIP is slightly slower than Baseline on Traces 1 and 3.

**Task 2 results**

| Policy | Trace | IPC | Speedup | L2 load MPKI | Instruction MPKI | Data MPKI |
|---|---|---|---|---|---|---|
| Baseline | 1 | 0.232 | 1.000 | 17.63 | 10.31 | 7.33 |
| Baseline | 2 | 0.250 | 1.000 | 28.80 | 20.24 | 8.56 |
| Baseline | 3 | 0.382 | 1.000 | 3.96 | 1.13 | 2.83 |
| SRRIP | 1 | 0.232 | 0.999 | 17.59 | 10.52 | 7.07 |
| SRRIP | 2 | 0.251 | 1.004 | 26.87 | 18.90 | 7.97 |
| SRRIP | 3 | 0.382 | 0.999 | 4.05 | 1.20 | 2.85 |
| I-RRIP | 1 | 0.232 | 0.999 | 17.59 | 9.88 | 7.71 |
| I-RRIP | 2 | 0.256 | 1.023 | 23.50 | 14.29 | 9.21 |
| I-RRIP | 3 | 0.382 | 1.000 | 3.91 | 1.03 | 2.88 |
| Dynamic I-RRIP | 1 | 0.232 | 1.000 | 17.59 | 9.89 | 7.70 |
| Dynamic I-RRIP | 2 | 0.256 | 1.022 | 23.66 | 14.52 | 9.14 |
| Dynamic I-RRIP | 3 | 0.382 | 1.000 | 3.91 | 1.03 | 2.88 |

**Dynamic I-RRIP: five-threshold sensitivity**

| Threshold | Trace 1 speedup | Trace 2 speedup | Trace 3 speedup | Geometric mean |
|---|---|---|---|---|
| 0.10 | 0.999 | 1.022 | 1.000 | 1.007 |
| 0.25 | 0.999 | 1.022 | 1.000 | 1.007 |
| 0.40 | 1.000 | 1.020 | 1.000 | 1.007 |
| 0.55 | 0.999 | 1.017 | 0.999 | 1.005 |
| 0.70 | 0.999 | 1.004 | 0.999 | 1.001 |

## Task 3: SPP + Prefetch-aware Dynamic I-RRIP

We keep the Task 1 SPP and add prefetch handling to Dynamic I-RRIP. In follower sets, we insert a data prefetch at RRPV 1, leave its RRPV unchanged on another prefetch hit, and promote it normally on a demand hit. A repeated prediction is not proof of demand reuse. Leader sets keep SRRIP, and prefetches do not train the decision counters. Insertion at 1 gave the best aggregate IPC among the tested insertion values; changing the decision threshold, leader count, window, or prefetch-hit promotion did not improve it. The geometric-mean speedup is 1.010 over Baseline, 1.007 over SPP, and 1.003 over Dynamic I-RRIP. It beats both standalone designs on Trace 2, but is 0.039% below SPP on Trace 1 and 0.003% below it on Trace 3. Thus it improves the aggregate result but does not meet the target on every trace.

**SPP + Prefetch-aware Dynamic I-RRIP results**

| Trace | IPC | Speedup | Accuracy (%) | Coverage (%) | L1D MPKI | L2 MPKI | Instruction MPKI | Data MPKI |
|---|---|---|---|---|---|---|---|---|
| 1 | 0.233 | 1.004 | 64.74 | 4.15 | 10.24 | 17.16 | 9.90 | 7.25 |
| 2 | 0.256 | 1.024 | 66.47 | 0.89 | 13.42 | 23.28 | 14.22 | 9.06 |
| 3 | 0.383 | 1.002 | 29.85 | 1.68 | 3.35 | 3.79 | 1.03 | 2.75 |

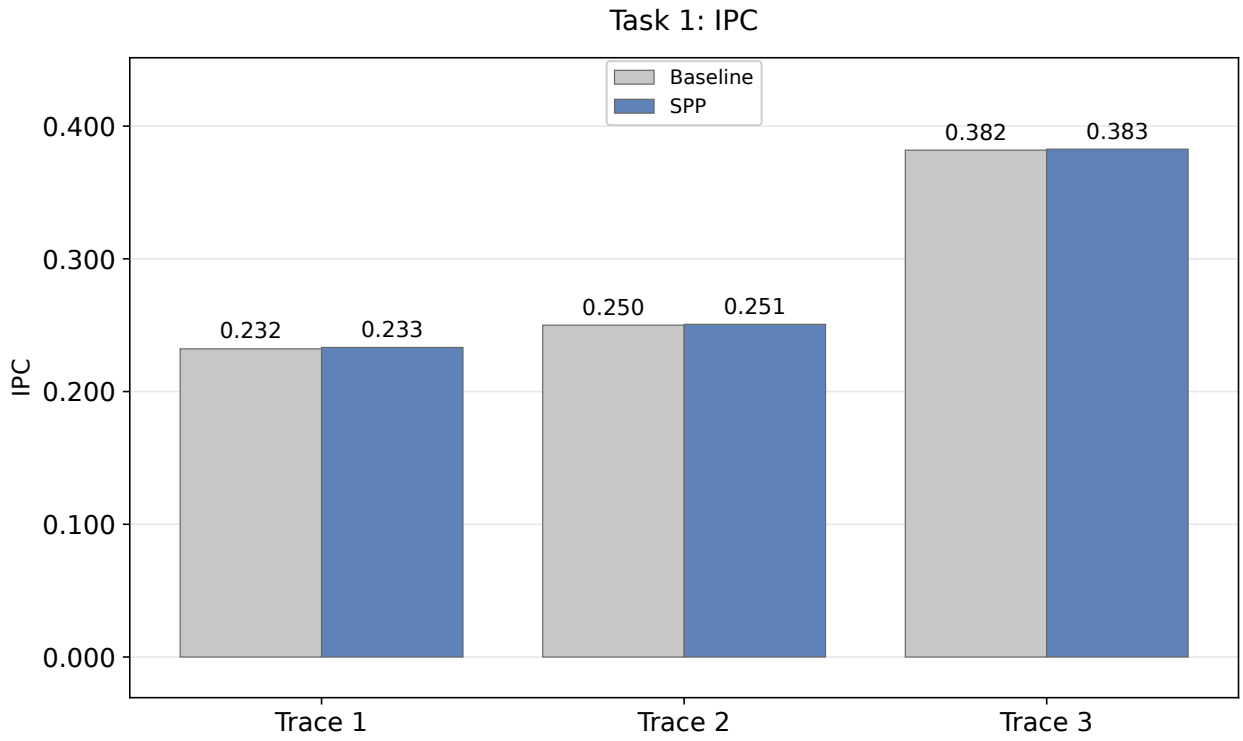



**Figure 1:** Task 1: IPC

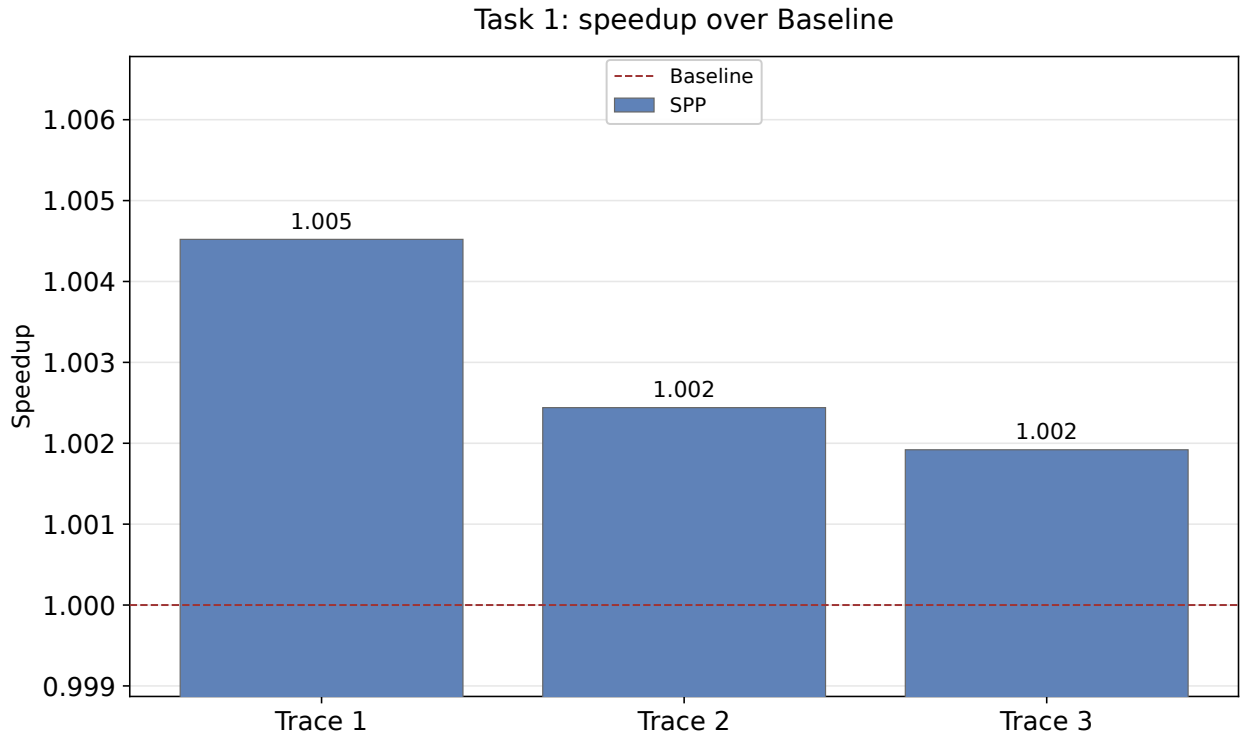



**Figure 2:** Task 1: speedup

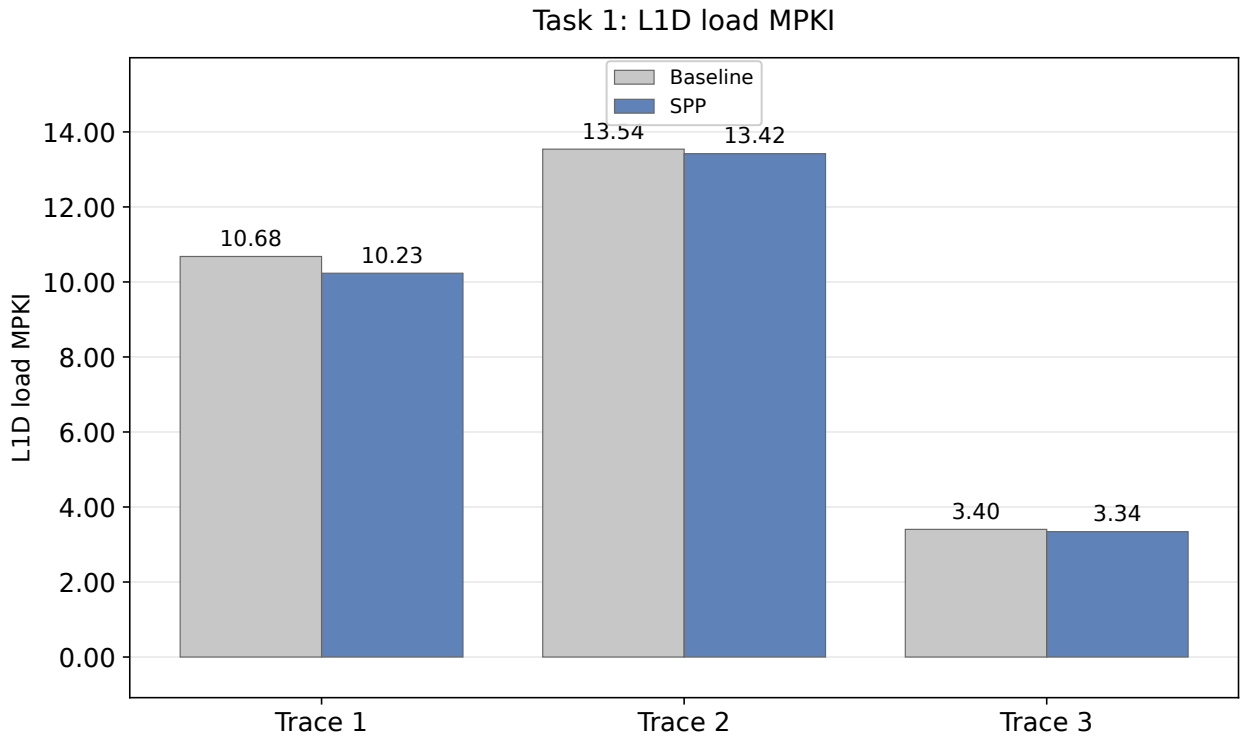



**Figure 3:** Task 1: L1D load MPKI

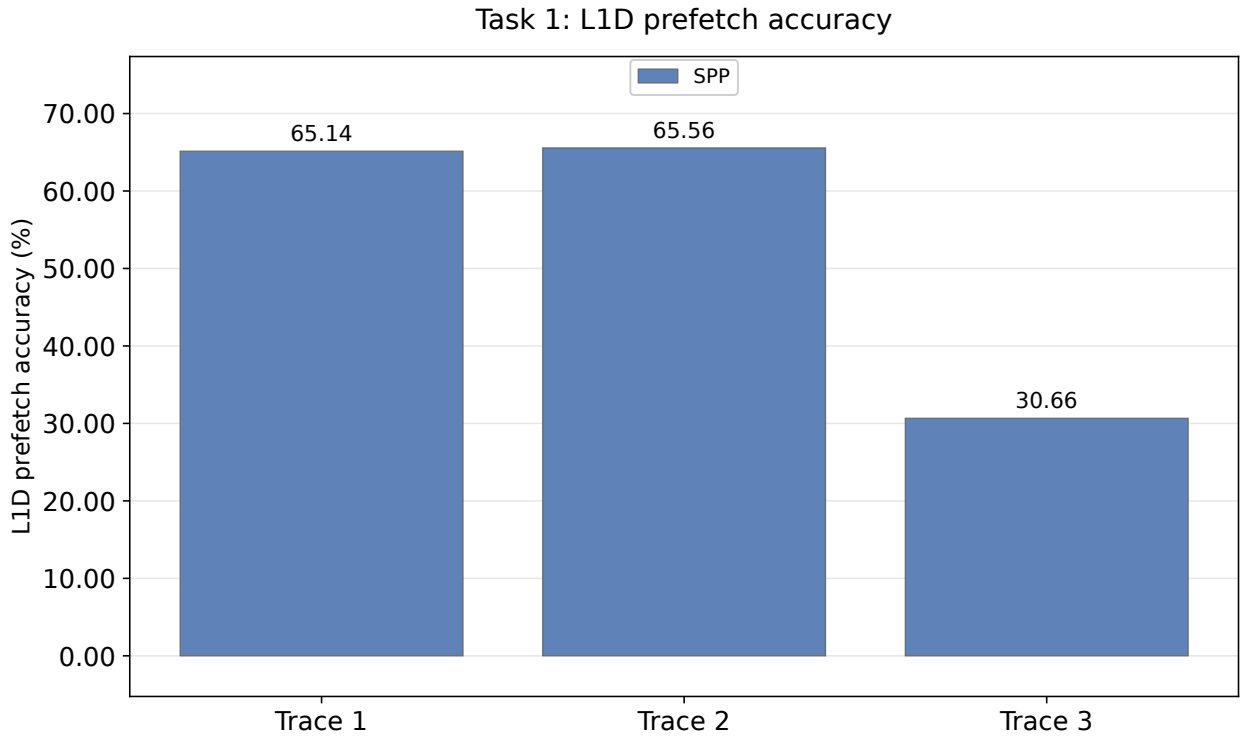



**Figure 4:** Task 1: prefetch accuracy

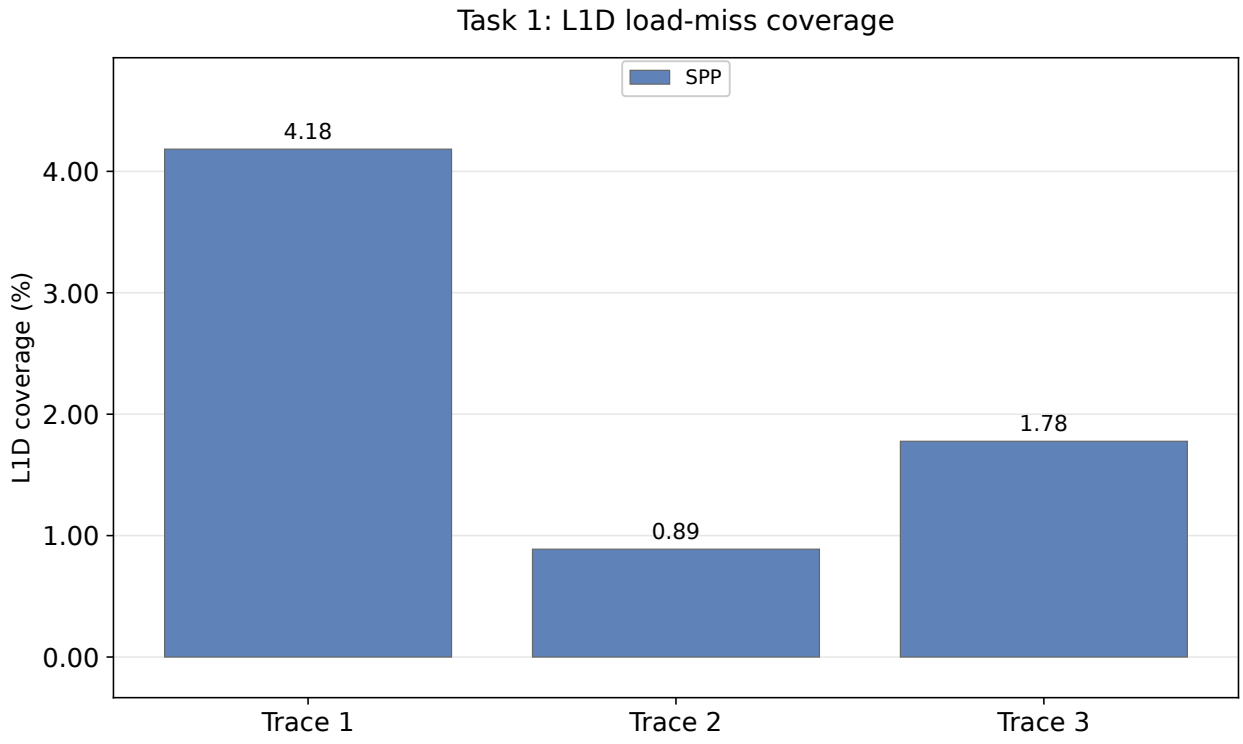



**Figure 5:** Task 1: L1D coverage

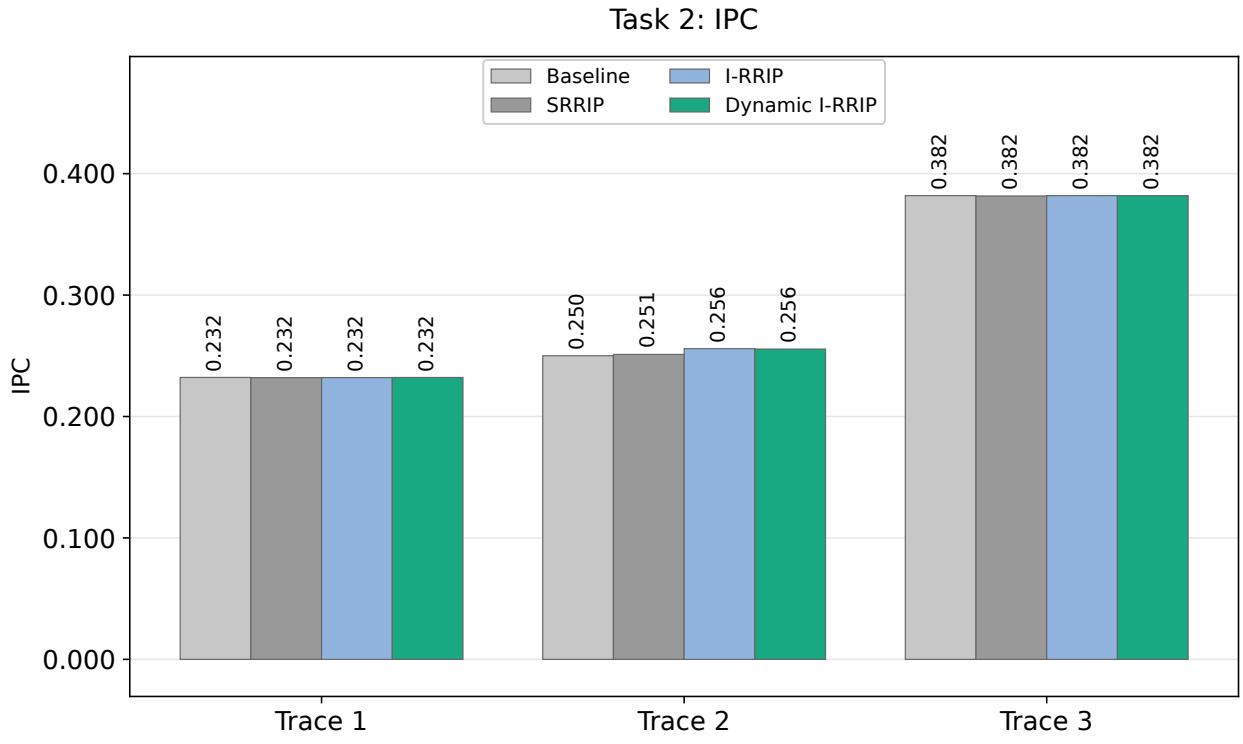



**Figure 6:** Task 2: IPC

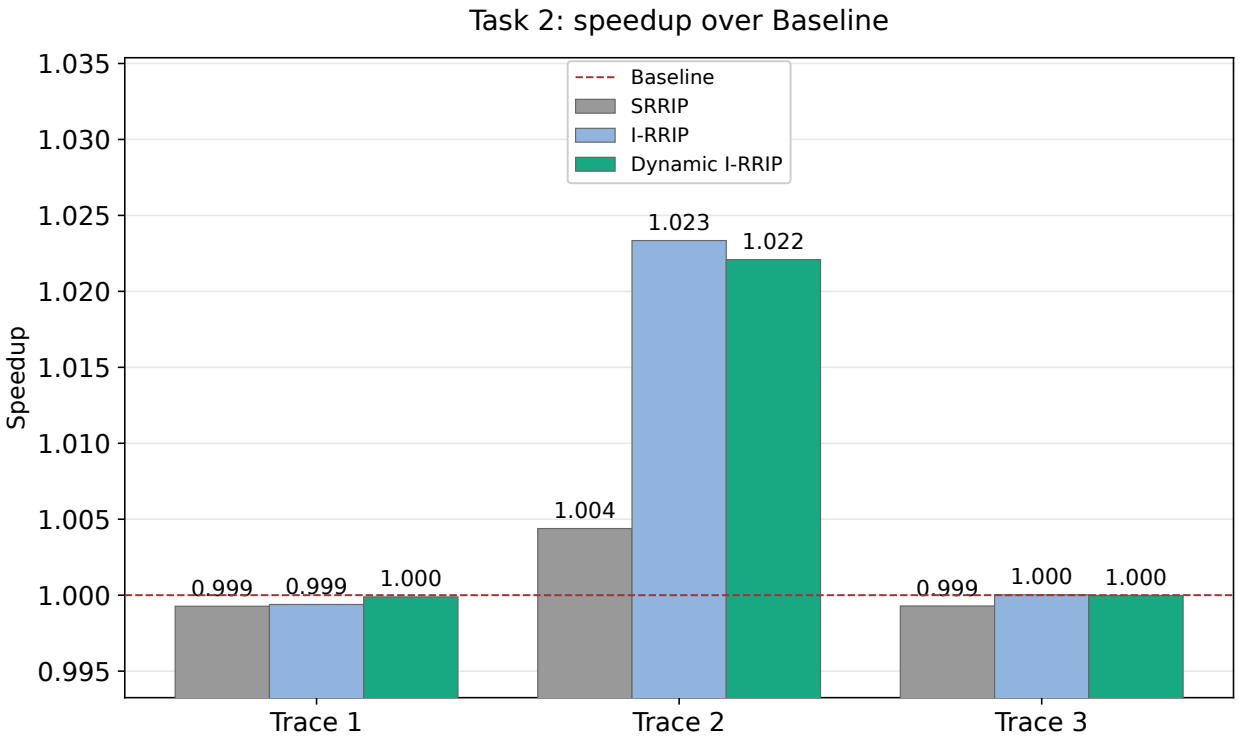



**Figure 7:** Task 2: speedup

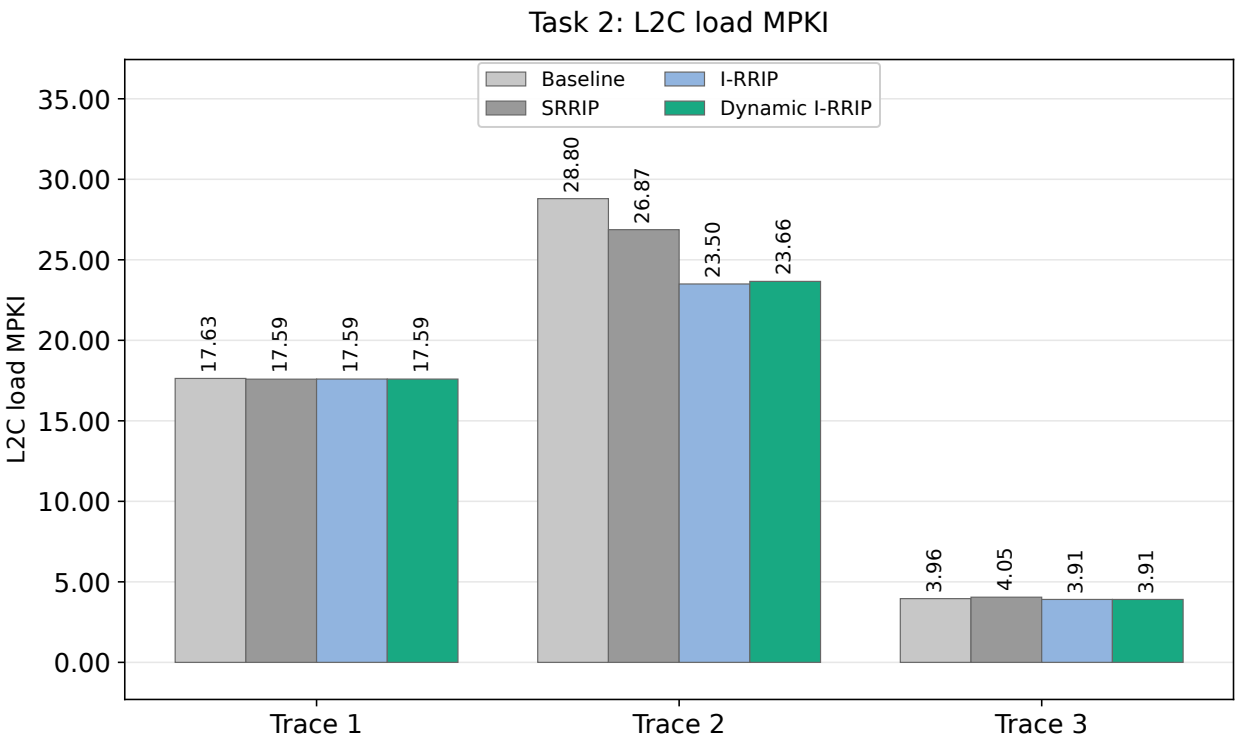



**Figure 8:** Task 2: L2 load MPKI

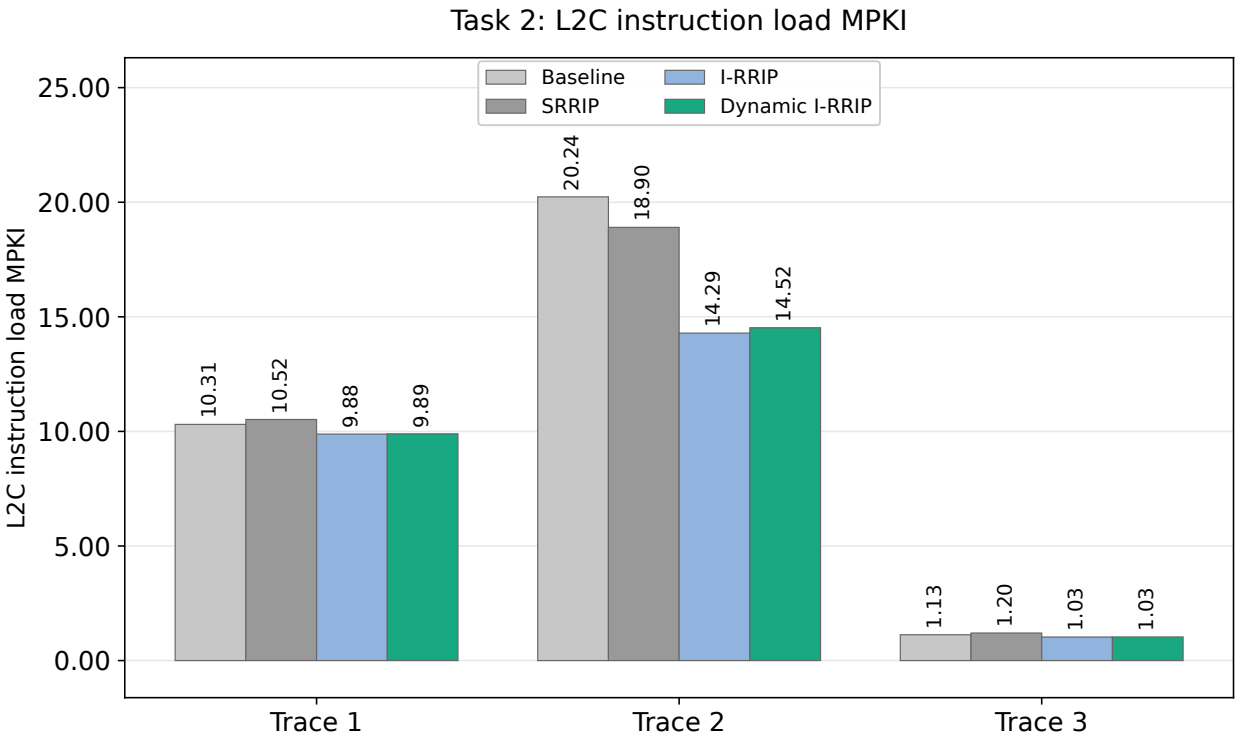



**Figure 9:** Task 2: instruction MPKI

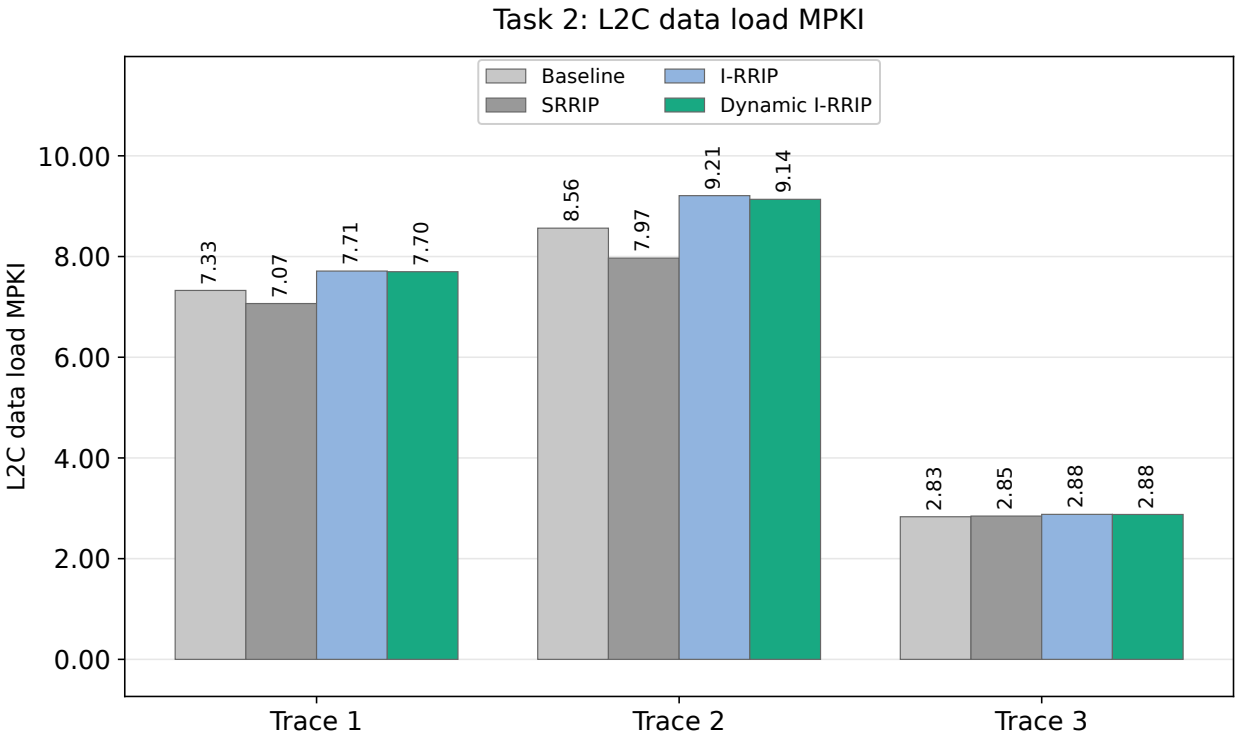



**Figure 10:** Task 2: data MPKI

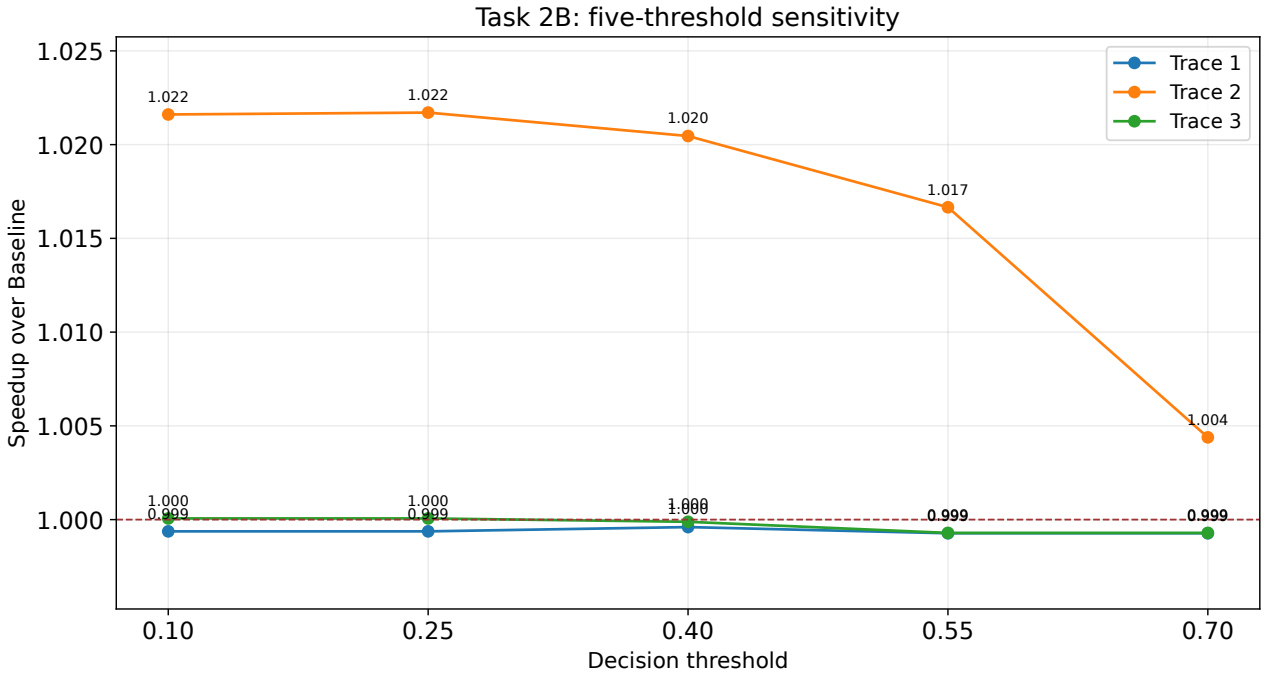



**Figure 11:** Task 2: threshold sensitivity

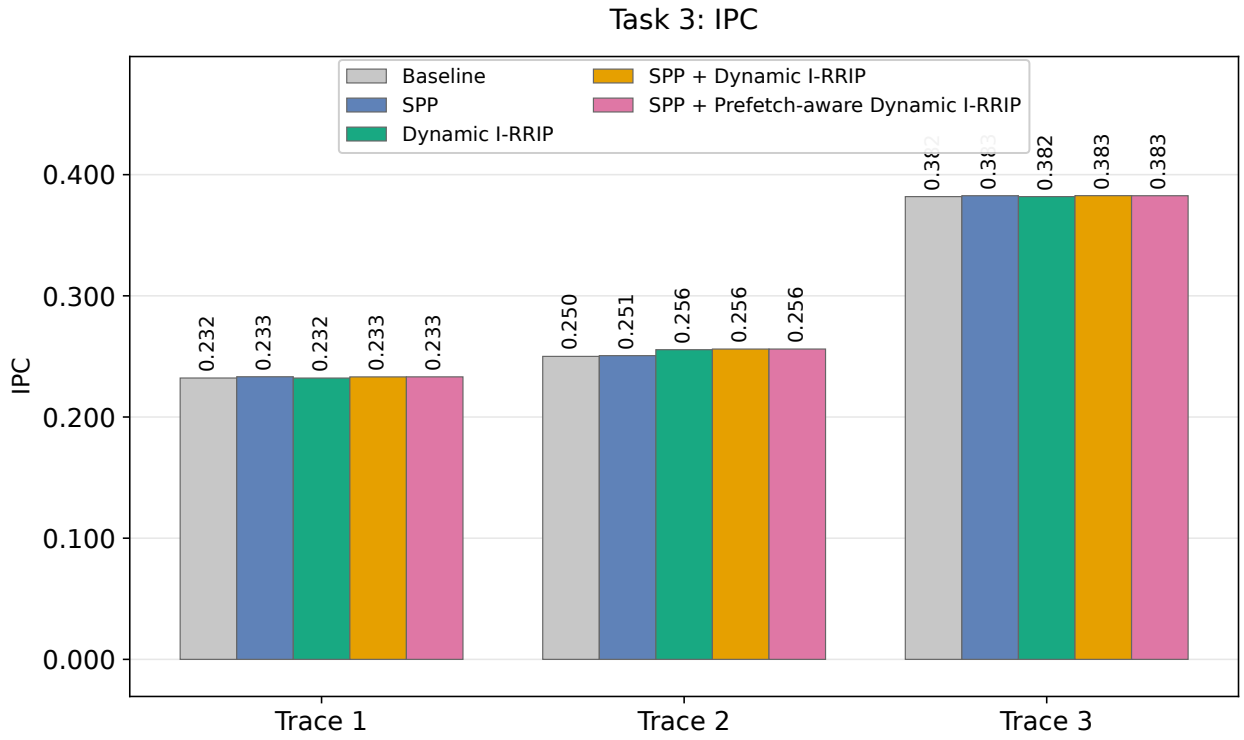



**Figure 12:** Task 3: IPC

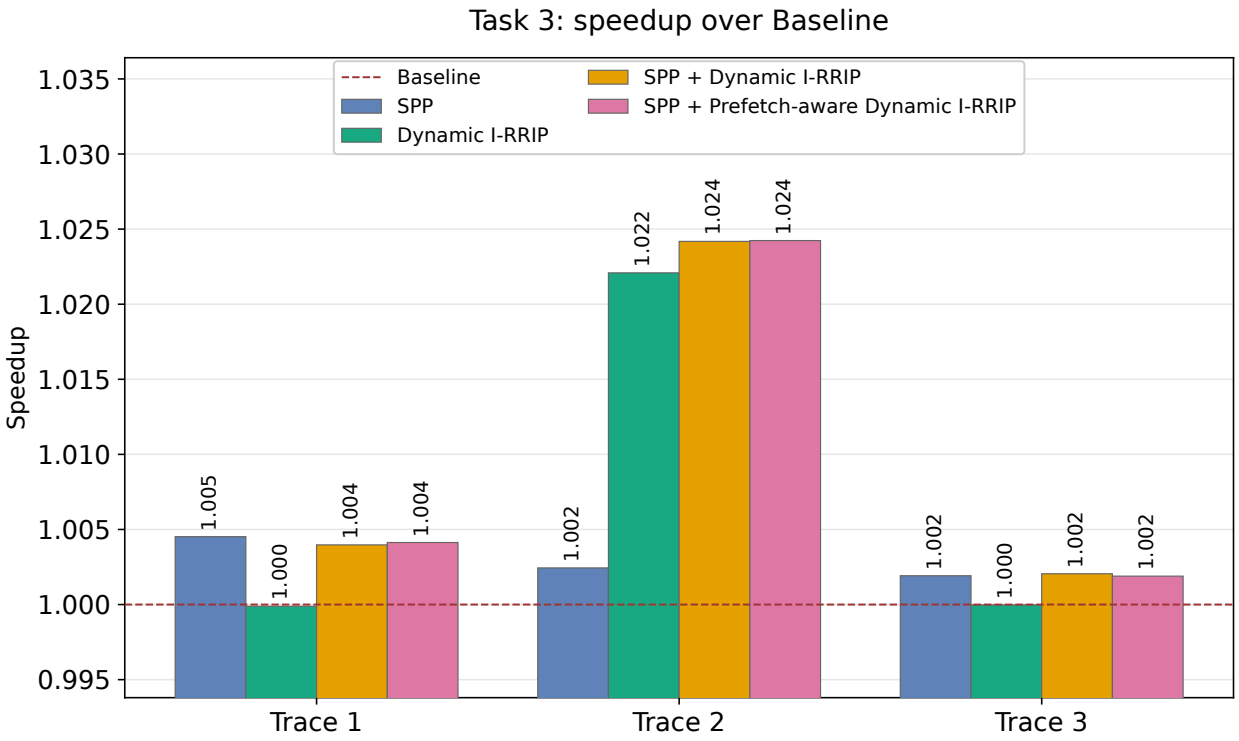



**Figure 13:** Task 3: speedup

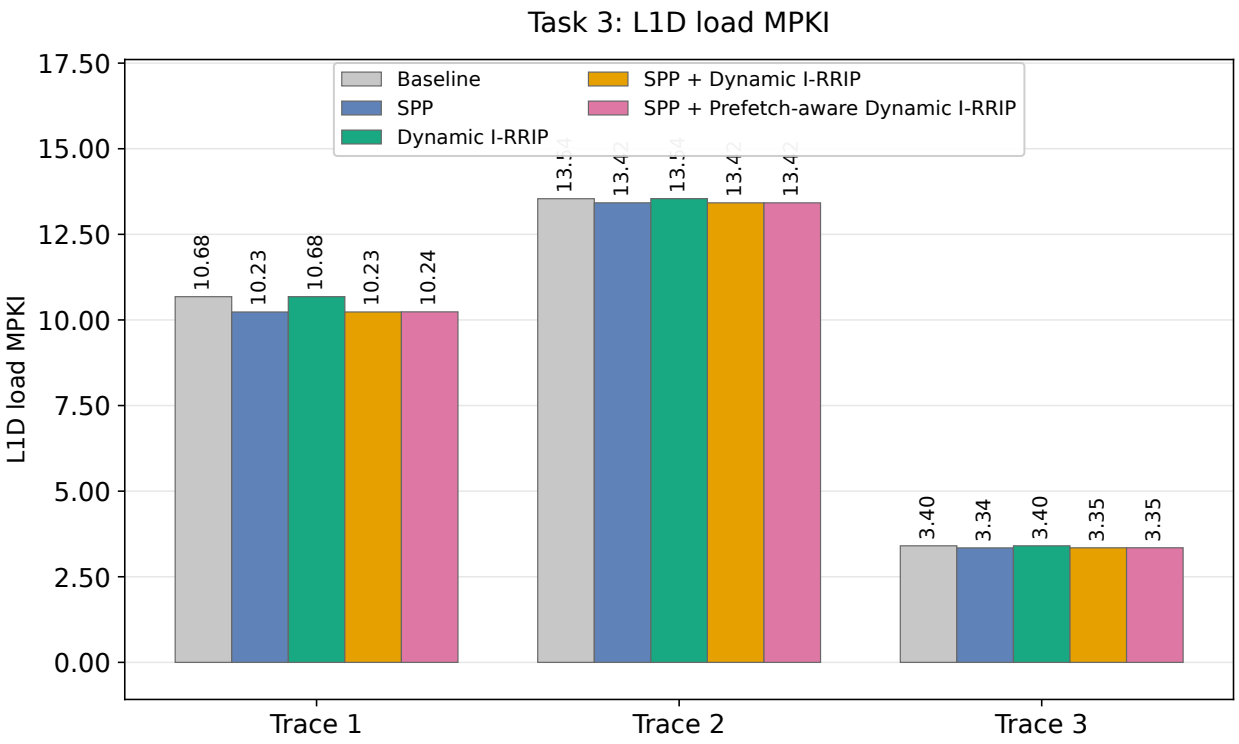



**Figure 14:** Task 3: L1D load MPKI

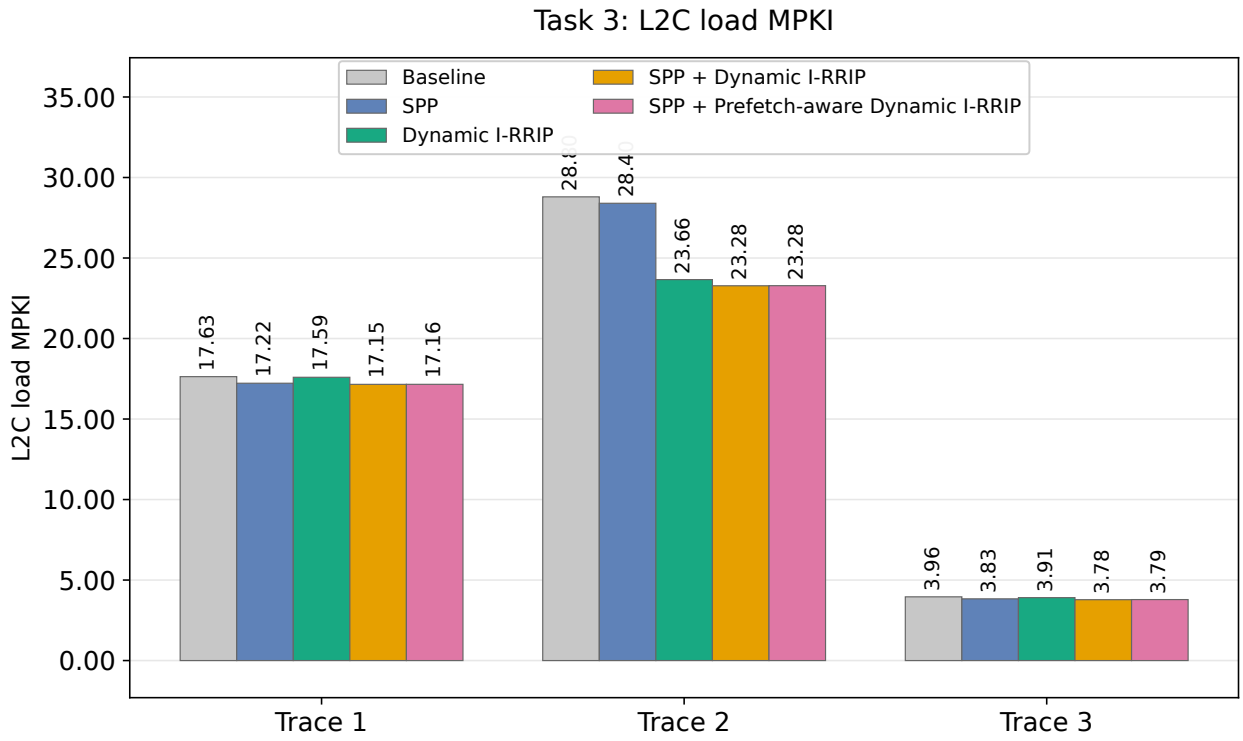



**Figure 15:** Task 3: L2 load MPKI

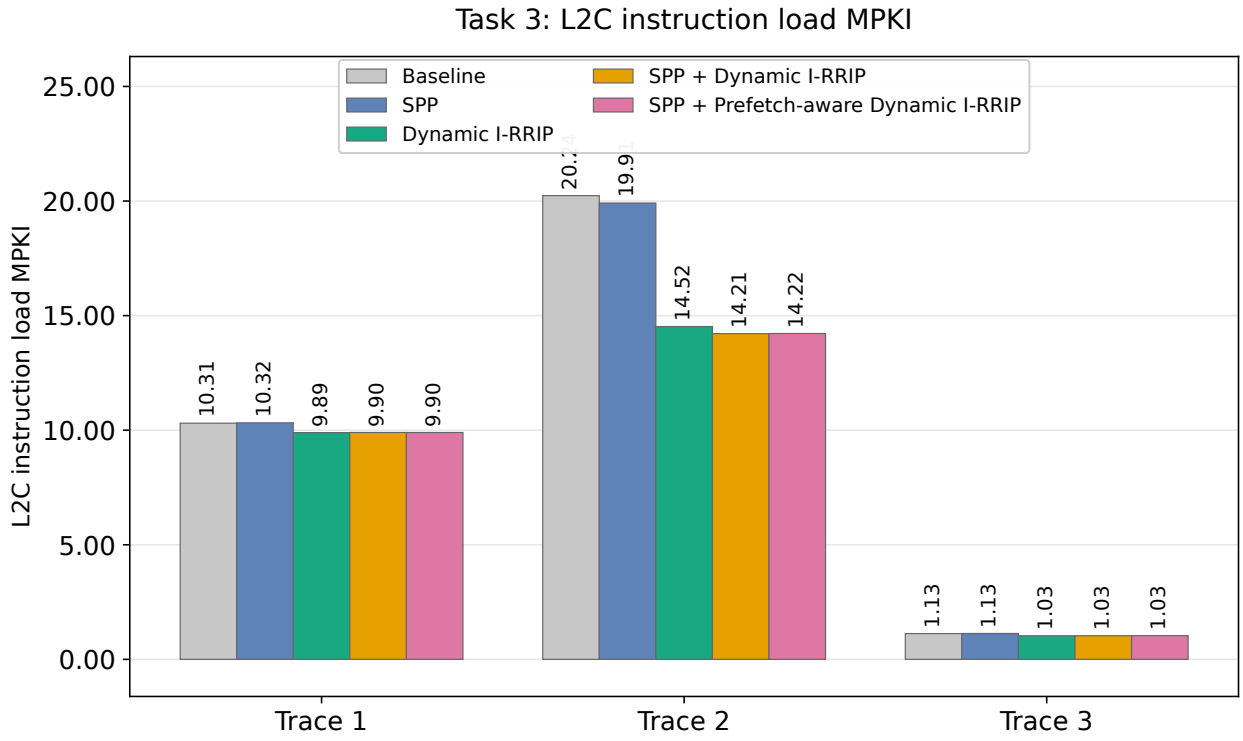



**Figure 16:** Task 3: instruction MPKI

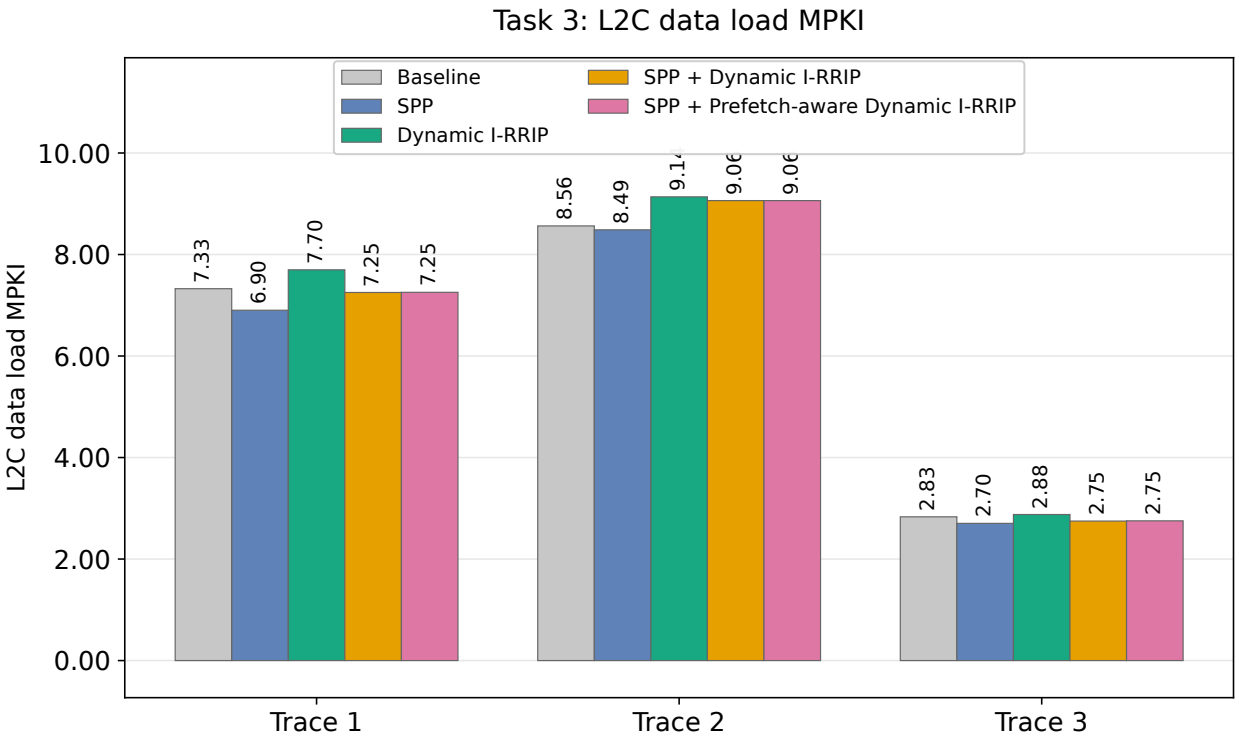



**Figure 17:** Task 3: data MPKI

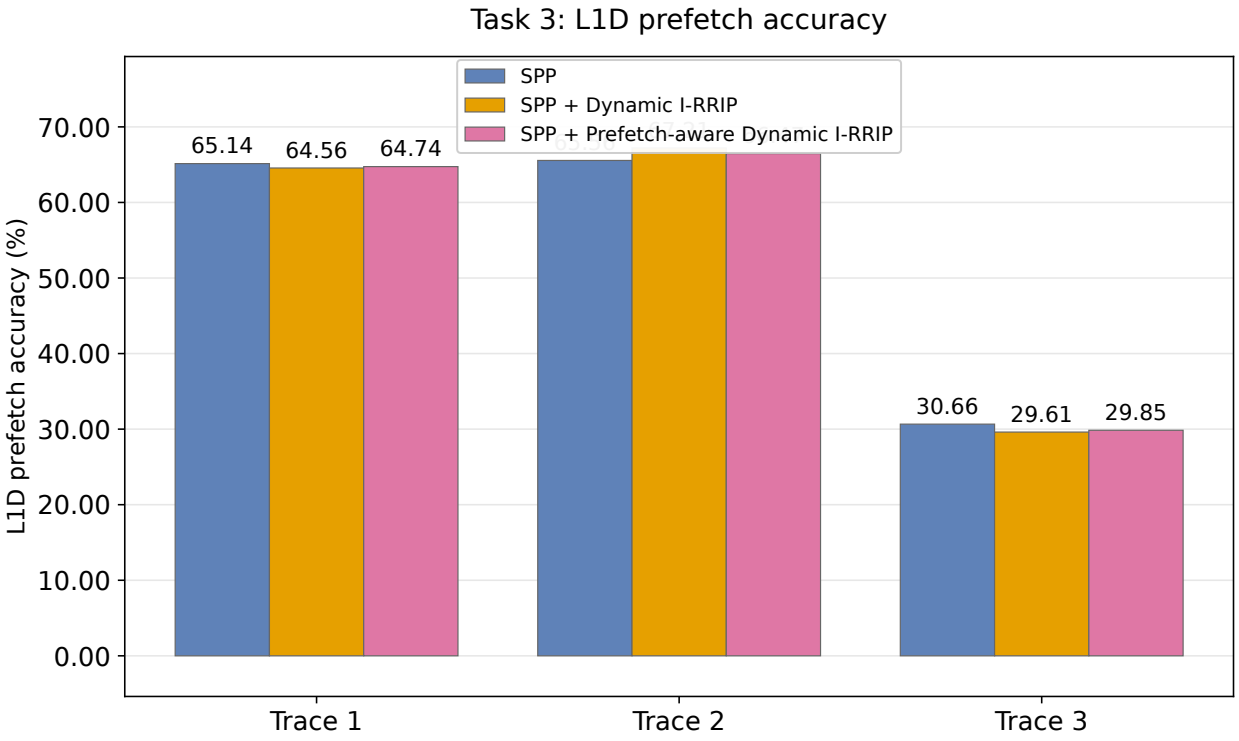



**Figure 18:** Task 3: prefetch accuracy

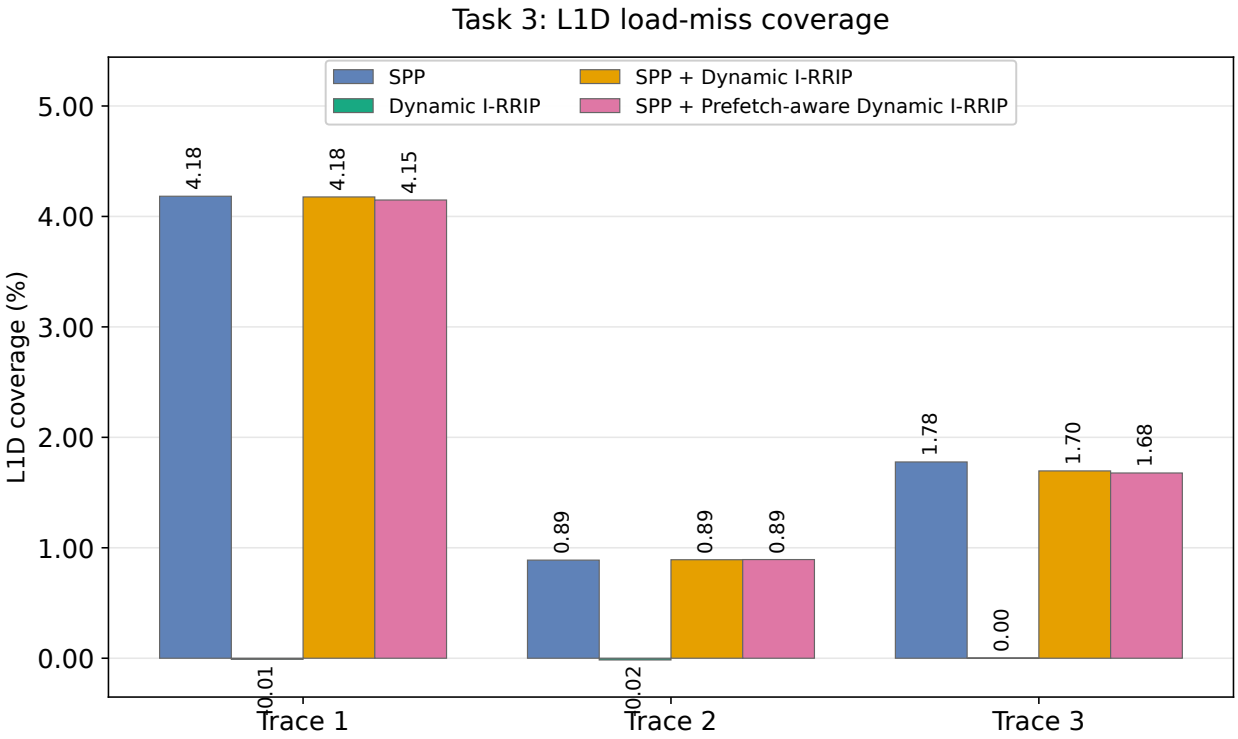



**Figure 19:** Task 3: L1D coverage